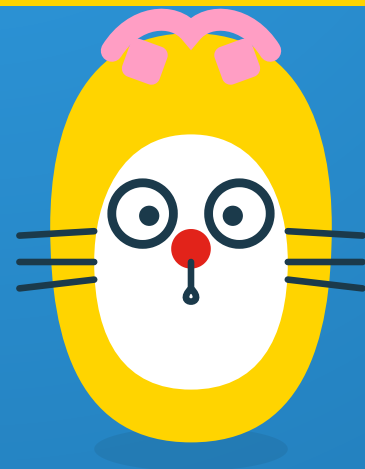


Git / GitHubを学ぶ

Gitは手元の作業を記録する道具。

GitHubは、その記録をみんなで共有する場所。



今回のゴール

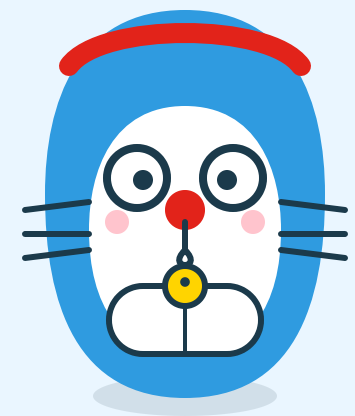
- GitとGitHubの違いを説明できる
- ローカルとGitHubの関係を理解する
- ブランチとマージの考え方が分かる
- commit / push / pullの基本操作が分かる

目次

1. GitとGitHubの違い
2. ローカルとGitHubの関係
3. ブランチとマージ
4. commit / push / pullの基本

01

GitとGitHubの違い



Git

- 自分のPCの中で、**ファイルの変更履歴を記録する**ソフト
- いつ・何を変えたかを、あとから追える
- インターネットに繋がってなくても使える

GitHub

- Gitの記録を**インターネット上に保管・共有する**サービス
- 他の人と同じRepositoryを見たり、変更を送り合ったりできる
- Gitがなければ動かないが、Git自体はGitHubがなくても使える

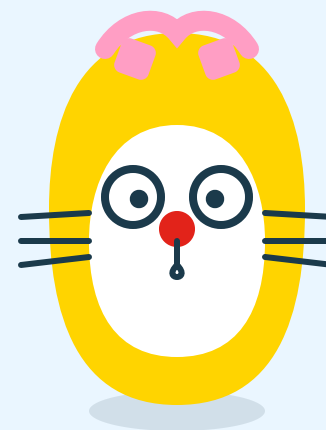


ポイント

Git = 自分のメモ帳の下書き。GitHub = その下書きを置く共有の本棚。

02

ローカルとGitHubの関係



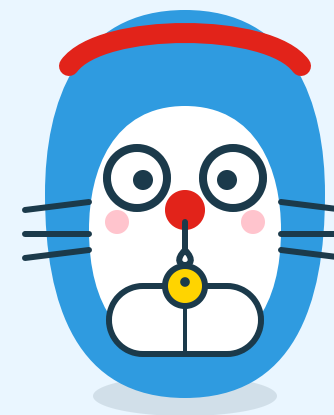
手元とクラウドの2つの場所

- **ローカル**：自分のPCの中にあるRepository
- **リモート (GitHub)**：クラウド上にある同じRepositoryのコピー
- 両者は「送る (push)」 「受け取る (pull)」 でやり取りする

```
ローカル (自分のPC)  --push--> GitHub  
ローカル (自分のPC)  <--pull-- GitHub
```

03

ブランチとマージ



ブランチとは

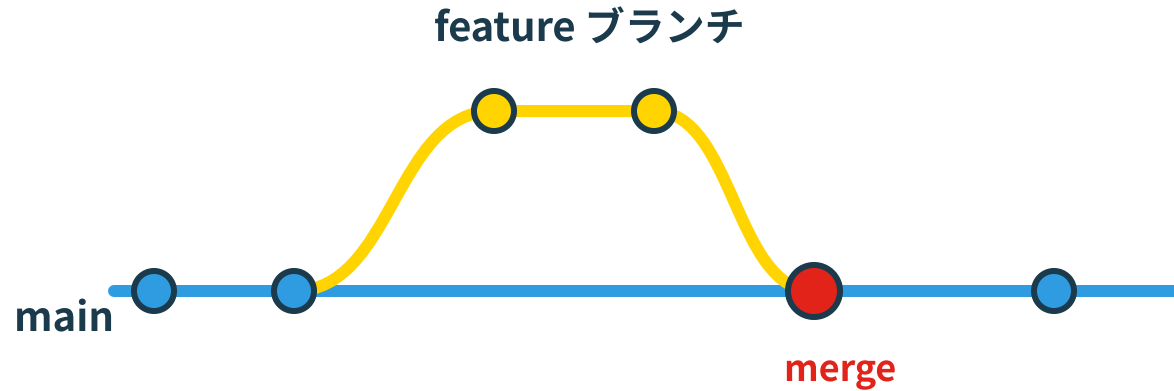
- 履歴の流れを**枝分かれさせて**、本番（`main`）に影響を与えずに作業する仕組み
- 資料の下書きを別コピーで進めるイメージに近い
- 作業が終わったら、`main`に合流させる。これを**マージ**と呼ぶ



ポイント

ブランチを切っておけば、試した作業が気に入らなくても`main`はそのまま無事。

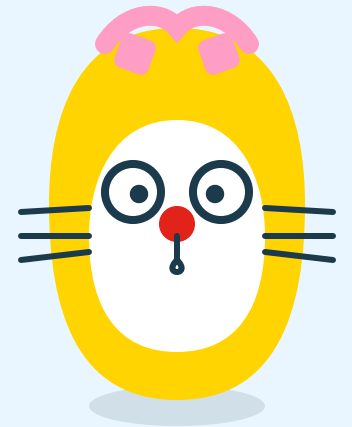
ブランチとマージの流れ（図解）



- **main** から枝分かれして **feature** ブランチで作業する
- 作業が終わったら、**main** にマージして合流する

04

commit / push / pullの基本



3つの基本操作

操作	することの一言
<code>commit</code>	変更内容を1つの記録として保存する
<code>push</code>	ローカルの記録をGitHubへ送る
<code>pull</code>	GitHubの記録をローカルへ取り込む



ガキ大将の注意報

commitしていない変更はpushできない。まず手元で記録してから送る、という順番を忘れないこと。

Second BrainをGitHubにpushする

1. 変更したファイルを確認する (`git status`)
2. 変更を記録する (`git add` → `git commit`)
3. GitHubへ送る (`git push`)



ひみつ道具メモ

パスワードや個人情報が入ったファイルは、そもそもcommitしない。一度送ると、記録には残り続ける。

次回は「AIにPC作業を任せる」を学びます



第6回（最終回）：AIにPC作業を任せる／自分専用の助手を作る